

DÉVELOPPEMENT D'UN SYSTÈME INTELLIGENT DE COLLECTE ET DE CENTRALISATION DES OFFRES D'EMPLOI TECH

CAS D'UTILISATION: WORKFINDER

Institut Universitaire de Technologie (IUT) Douala
Département de Génie Informatique
Année Académique : 2025-2026

INFORMATIONS GÉNÉRALES

Titre du Projet : WorkFinder – Plateforme intelligente d'agrégation d'offres d'emploi

Membres du Groupe :

- [Nom et Prénom Étudiant 1] - [Matricule]
- [Nom et Prénom Étudiant 2] - [Matricule]
- [Nom et Prénom Étudiant 3] - [Matricule]
- [Nom et Prénom Étudiant 4] - [Matricule]
- [Nom et Prénom Étudiant 5] - [Matricule]
- Filière : Génie Logiciel (Licence 3)

Encadrement :

- Encadreur Académique : [Nom et titre]
- Encadreur Professionnel (si applicable) : [Nom et entreprise]

Durée du Projet : 14 Semaines

1. CONTEXTE ET JUSTIFICATION DU PROJET DE SYSTÈME INTELLIGENT DE COLLECTE ET DE CENTRALISATION

1.1 Contexte Général

Le marché de l'emploi au Cameroun, et particulièrement dans le secteur technologique, est caractérisé par une forte fragmentation de l'information. Les offres d'emploi sont dispersées sur de nombreux canaux : sites web spécialisés, portails locaux, réseaux sociaux, flux RSS et APIs diverses. Cette dispersion rend la veille technologique et la recherche d'emploi fastidieuses pour les candidats.

1.2 Problématique

Actuellement, la recherche d'emploi au Cameroun s'avère être un processus particulièrement complexe et inefficace pour les candidats.

- **Fragmentation de l'information** : Les offres d'emploi sont dispersées sur une multitude de canaux hétérogènes (sites web spécialisés, réseaux sociaux, affichage physique), rendant une veille exhaustive difficile.
- **Redondance de l'information** : Les candidats sont confrontés à une duplication massive des annonces sur différentes plateformes, augmentant le bruit informationnel.
- **Inefficacité du processus** : Le temps considérable consacré au filtrage manuel des offres non pertinentes ou obsolètes se fait au détriment de la préparation des candidatures.

Synthèse : Bien que l'information soit disponible, son manque de structuration et de centralisation constitue un obstacle majeur à l'efficacité du marché de l'emploi.

1.3 Justification

WorkFinder se positionne comme un **système intelligent de collecte et de centralisation** et d'agrégation d'offres d'emploi centralisé, spécifiquement adapté au contexte local. Le système propose un changement de paradigme : l'automatisation de la collecte d'information remplace la recherche manuelle fastidieuse.

1. **Collecte Automatisée** : Agrégation continue des offres provenant des sources majeures du marché.
2. **Traitement et Normalisation** : Nettoyage des données, élimination des doublons et structuration standardisée.
3. **Centralisation** : Mise à disposition d'un point d'accès unique et cohérent.

Bénéfice : Le candidat accède instantanément à une vue exhaustive et épurée du marché, optimisant ainsi sa recherche. Ce projet permet également d'appliquer des compétences avancées en ingénierie logicielle (backend, scraping, algorithmes de matching, frontend moderne).

1.4 Bénéficiaires

- **Chercheurs d'emploi** : Particulièrement les profils Tech au Cameroun, qui gagneront un temps précieux.
- **Recruteurs et entreprises** : Qui bénéficieront d'une visibilité accrue de leurs offres centralisées.
- **Administrateurs de la plateforme** : Qui pourront analyser les tendances du marché de l'emploi local.

2. OBJECTIFS DU PROJET

2.1 Objectif Général

Créer une plateforme complète et un **système intelligent de collecte et de centralisation des offres d'emploi tech** qui collecte automatiquement les offres d'emploi depuis de multiples sources, les normalise, les dédoublonne et les propose aux utilisateurs via une interface moderne avec recherche avancée et alertes personnalisées.

2.2 Objectifs Spécifiques

Les objectifs spécifiques de ce projet sont :

1. **Collecte Automatisée** : Ingérer les offres depuis 5 sources identifiées (APIs, RSS, sites HTML locaux).
2. **Traitement des Données** : Nettoyer et normaliser les données, et éliminer les doublons via un algorithme de hashing et fuzzy matching.

- 3. **Stockage Performant** : Stocker les données dans PostgreSQL avec indexation textuelle pour une recherche rapide.
- 4. **Expérience Utilisateur** : Fournir un frontend moderne (React) et un moteur de recherche multi-critères.
- 5. **Système d'Alerte** : Notifier les utilisateurs via Email pour les nouvelles offres pertinentes.

3. ANALYSE DE L'EXISTANT

3.1 Solutions Existantes

Solution	Points Forts	Points Faibles	Coût
Jobartis CM / Emploi.cm	Contenu local riche.	Interface parfois datée, pas d'agrégation d'autres sources, recherche limitée.	Gratuit / Premium
LinkedIn Jobs	Réseau professionnel, volume d'offres.	Mélange d'offres non pertinentes, fonctionnalités premium payantes, pas spécifique au contexte local tech.	Freemium
Agrégateurs généralistes (Indeed)	Volume énorme.	Beaucoup de doublons, scraping souvent bloqué, peu adapté aux spécificités locales.	Gratuit (Ads)

3.2 CRITIQUE DE L'EXISTANT

Les solutions actuelles sont soit trop généralistes et mondiales (ne captant pas bien le marché local), soit trop locales et limitées techniquement (pas d'agrégation externe, UX pauvre). Le problème majeur reste la **duplication** et la **difficulté de filtrage**. Aucune solution ne centralise efficacement les offres tech camerounaises dispersées.

3.3 Positionnement du Projet

WorkFinder se positionne comme un **agrégateur intelligent et un système de centralisation spécialisé**. Contrairement aux sites d'emploi classiques, il ne se contente pas d'afficher des annonces postées manuellement, mais va activement chercher l'information ailleurs pour la centraliser, tout en offrant une expérience utilisateur supérieure (UX moderne, alertes instantanées).

4. SPÉCIFICATIONS FONCTIONNELLES

4.1 Fonctionnalités Principales

4.1.1 Ingestion et Normalisation (Backend)

Description : Le cœur du **système intelligent de collecte et de centralisation** assure la collecte automatique des offres depuis Remoteive, Adzuna, WeWorkRemotely, Jobartis et Emploi.cm. **Acteurs concernés** : Système (Scheduler), Administrateur. **Données entrantes** : Flux RSS, Réponses API JSON, Pages HTML brutes. **Données sortantes** : Objets "Job" normalisés et dédoublonnés en base de données. **Règles de gestion** :

- Parsing spécifique par source.
- Extraction des compétences (skills) via regex/dictionnaires.
- Dédoublonnage via Hash (SHA1 sur titre+entreprise+date) et Fuzzy Matching (RapidFuzz, seuil > 90% titre, > 85% entreprise).

4.1.2 Recherche et Consultation (Frontend)

Description : Interface de recherche d'offres d'emploi permettant aux utilisateurs de trouver des opportunités. **Acteurs concernés :** Utilisateur (anonyme ou connecté). **Données entrantes :** Mots-clés, Filtres (Compétences, Remote, Localisation). **Données sortantes :** Liste d'offres paginée, Détails d'une offre. **Règles de gestion :**

- Recherche textuelle (Full-text search) rapide (< 200ms).
- Affichage clair des détails (salaire, date, lien original).

4.1.3 Gestion des Alertes (Watchlists)

Description : Permettre aux utilisateurs de créer des alertes sur des critères spécifiques pour recevoir des notifications. **Acteurs concernés :** Utilisateur connecté. **Données entrantes :** Critères d'alerte (Mots-clés, Ville), Fréquence. **Données sortantes :** Notifications Email. **Règles de gestion :**

- Canaux : Email uniquement (pour cette version).
- Pas de doublon de notification pour la même offre.

4.1.4 Administration

Description : Dashboard pour gérer les sources et surveiller l'ingestion. **Acteurs concernés :** Administrateur. **Données entrantes :** Commandes d'ingestion, Configuration des sources. **Données sortantes :** État des services (Health check), Logs d'ingestion. **Règles de gestion :** Accès sécurisé réservé aux administrateurs.

4.2 Fonctionnalités Secondaires

1. **Gestion de profil utilisateur :** Modification des informations personnelles (Email, Mot de passe).
2. **Historique de recherche :** Sauvegarde des dernières recherches effectuées.
3. **Favoris :** Possibilité de marquer des offres comme favorites sans créer d'alerte.

4.3 Cas d'Utilisation (Use Cases)

Cas d'utilisation 1 : Consulter les offres

- **Acteur principal :** Utilisateur (anonyme ou connecté)
- **Préconditions :** Le site est accessible.
- **Scénario nominal :**
 1. L'utilisateur accède à la page d'accueil.
 2. Il saisit des mots-clés ou sélectionne des filtres.
 3. Le système affiche les offres correspondantes triées par pertinence.
 4. L'utilisateur clique sur une offre pour voir les détails.
- **Scénarios alternatifs :** Aucune offre trouvée (affichage message vide).
- **Postconditions :** L'utilisateur a pris connaissance des offres.

Cas d'utilisation 2 : Créer une alerte (Watchlist)

- **Acteur principal** : Utilisateur connecté
- **Préconditions** : L'utilisateur est authentifié.
- **Scénario nominal** :
 1. L'utilisateur définit ses critères (ex: "Python", "Douala").
 2. Il clique sur "Créer une alerte".
 3. Le système enregistre la watchlist.
 4. À chaque nouvelle ingestion, le système vérifie les correspondances et notifie l'utilisateur par email.
- **Scénarios alternatifs** : L'utilisateur a déjà atteint le nombre max d'alertes.
- **Postconditions** : L'alerte est active.

Cas d'utilisation 3 : Ingestion des données

- **Acteur principal** : Scheduler / Admin
- **Préconditions** : Les sources externes sont accessibles.
- **Scénario nominal** :
 1. Le Cron (ou l'admin) lance le fetcher.
 2. Le système récupère les pages/API.
 3. Le système parse le contenu et normalise les données.
 4. Le système vérifie les doublons (Hash/Fuzzy).
 5. Le système insère les nouvelles offres en base.
- **Scénarios alternatifs** : Erreur de connexion à une source (log de l'erreur et passage à la suivante).
- **Postconditions** : La base de données est à jour.

5. SPÉCIFICATIONS TECHNIQUES

5.1 Architecture Système

5.1.1 Type d'Architecture

Le choix s'est porté sur une architecture de type **Monolithe Modulaire**. Ce modèle constitue une approche intermédiaire entre le monolithe traditionnel et les microservices, offrant une unité de déploiement unique mais une isolation logique stricte des modules (Ingestion, Recherche, Utilisateurs), idéale pour ce **système intelligent de collecte et de centralisation**.

5.1.2 Schéma d'Architecture

- **Frontend** : SPA React communiquant via API REST.
- **Backend** : FastAPI servant l'API et orchestrant les modules.
- **Workers** : Celery pour les tâches asynchrones (Scraping, Email).
- **Data** : PostgreSQL (Stockage) + Redis (Broker/Cache).

5.2 Technologies et Outils

5.2.1 Langages de Programmation

- **Backend** : Python 3.11+
- **Frontend** : TypeScript, HTML5, CSS3

5.2.2 Frameworks et Bibliothèques

- **Backend** : FastAPI (Web), Celery (Async), SQLAlchemy (ORM), RapidFuzz (Matching).
- **Frontend** : React (UI), Tailwind CSS (Styling), Axios (HTTP).

5.2.3 Base de Données

- **Type** : Relationnelle (avec support JSONB et Full-Text Search).
- **SGBD** : PostgreSQL.
- **Justification** : Robustesse, support natif du JSON pour les données brutes des offres, et performance de l'indexation textuelle (tsvector) évitant un moteur de recherche externe complexe pour le MVP.

5.2.4 Outils de Développement

- **IDE** : VS Code.
- **Gestion de version** : Git, GitHub.
- **Gestion de projet** : Trello / GitHub Projects.
- **Tests** : Pytest (Backend), Jest (Frontend).
- **Documentation** : Swagger/OpenAPI (généré par FastAPI).

5.2.5 Environnement de Déploiement

- **Conteneurisation** : Docker, Docker Compose.
- **CI/CD** : GitHub Actions (Tests automatiques + Build).
- **Hébergement** : Render, Railway ou VPS Linux.

5.3 Exigences Non Fonctionnelles

5.3.1 Performance

- Temps de réponse de la recherche : < 200ms.
- Capacité d'ingestion : Traitement de 1000+ offres par cycle sans blocage.

5.3.2 Sécurité

- **Authentification** : JWT (JSON Web Tokens).
- **Mots de passe** : Hachage fort (Argon2 ou bcrypt).
- **Protection** : Validation stricte des entrées (Pydantic), protection CORS.

5.3.3 Utilisabilité

- Interface Responsive (Mobile First).
- Design moderne et intuitif (Tailwind CSS).
- Feedback utilisateur clair (chargement, erreurs).

5.3.4 Fiabilité

- Gestion robuste des erreurs de scraping (ne pas crasher si une source change).
- Retries automatiques pour les tâches Celery échouées.

5.3.5 Maintenabilité

- Code modulaire (Monolithe Modulaire).
- Typage strict (TypeScript, Python Type Hints).
- Documentation du code et de l'API.

5.3.6 Portabilité

- Application entièrement conteneurisée (Docker), déployable sur n'importe quel environnement supportant Docker.

5.3.7 Évolutivité

- Architecture préparée pour une migration vers des microservices si la charge le nécessite.
- Ajout facile de nouvelles sources d'ingestion (Pattern Strategy).

6. MODÉLISATION

6.1 Diagrammes UML Requis

6.1.1 Diagramme de Cas d'Utilisation

Vue d'ensemble des interactions : Consulter, Filtrer, Voir détails, Auth (Login/Register), Gérer watchlists, Recevoir alertes, Lancer ingestion. Acteurs : Utilisateur, Admin, Système.

6.1.2 Diagramme de Classes

Structure statique : Classes **User**, **Watchlist**, **Job**, **Notification**, **Source**, **Parser**, **IngestionWorker**, **MatchingEngine** et leurs relations.

6.1.3 Diagramme de Séquence

Interactions temporelles pour :

1. Ingestion → Parsing → Dedup → Storage
2. User search → API → DB → Frontend
3. Watchlist match → NotificationEngine → Email Service

6.1.4 Diagramme d'Activité

Flux de travail de l'ingestion : Début -> Fetch Source -> Parse HTML -> Calcul Hash -> Existe en DB ? -> (Non) Insérer -> (Oui) Ignorer -> Fin. (Crucial pour le dédoublement).

6.1.5 Diagramme de Déploiement

Architecture physique : Frontend (Vercel/Netlify), Backend API (Render/VPS), Worker (Celery), DB (PostgreSQL), Broker (Redis).

6.2 Modèle de Données

6.2.1 Modèle Conceptuel de Données (MCD)

Le Modèle Conceptuel de Données décrit les entités du système et leurs relations, indépendamment de la technologie de stockage.

Entités Principales :

- **User** : Représente un utilisateur inscrit sur la plateforme.
 - Attributs : id, email, password, role, created_at.
- **Job** : Représente une offre d'emploi collectée.
 - Attributs : id, title, company, location, remote, description, skills, url, posted_date.
- **Watchlist** : Représente une alerte de recherche configurée par un utilisateur.
 - Attributs : id, keywords, frequency.
- **Source** : Représente une source de données (site web ou API).
 - Attributs : id, name, url, status.

Relations :

- **User - Watchlist** : Un utilisateur peut créer plusieurs watchlists (1,n). Une watchlist appartient à un seul utilisateur (1,1).
- **Job - Source** : Un job provient d'une source spécifique (1,1). Une source peut fournir plusieurs jobs (0,n).

6.2.2 Modèle Logique de Données (MLD)

Le MLD traduit le MCD en un schéma relationnel.

- **users** (#id, email, password_hash, role, created_at)
- **sources** (#id, name, url, status)
- **jobs** (#id, title, company, location, remote, description, skills, normalized_hash, raw_json, #source_id)
 - Clé étrangère : source_id référence sources(id)
- **watchlists** (#id, keywords, frequency, #user_id)
 - Clé étrangère : user_id référence users(id)

6.2.3 Modèle Physique de Données (MPD)

Le MPD décrit l'implémentation spécifique dans le SGBD PostgreSQL.

Table **users**

Colonne	Type	Contrainte	Description
id	SERIAL	PK	Identifiant unique
email	VARCHAR(255)	UNIQUE, NOT NULL	Email de l'utilisateur

Colonne	Type	Contrainte	Description
password_hash	VARCHAR(255)	NOT NULL	Hash du mot de passe
role	VARCHAR(50)	DEFAULT 'user'	Rôle (admin/user)
created_at	TIMESTAMP	DEFAULT NOW()	Date d'inscription

Table jobs

Colonne	Type	Contrainte	Description
id	SERIAL	PK	Identifiant unique
title	TEXT	NOT NULL	Titre de l'offre
company	TEXT	NOT NULL	Nom de l'entreprise
location	TEXT		Lieu du poste
remote	BOOLEAN	DEFAULT FALSE	Télétravail possible
description	TEXT		Description complète
skills	TEXT[]		Liste des compétences (Array)
normalized_hash	VARCHAR(64)	UNIQUE	Hash pour dédoublement
raw_json	JSONB		Données brutes de la source
source_id	INTEGER	FK -> sources(id)	Source de l'offre

Table watchlists

Colonne	Type	Contrainte	Description
id	SERIAL	PK	Identifiant unique
user_id	INTEGER	FK -> users(id)	Propriétaire de l'alerte
keywords	TEXT[]	NOT NULL	Mots-clés surveillés
frequency	VARCHAR(20)	DEFAULT 'daily'	Fréquence de notification

Table sources

Colonne	Type	Contrainte	Description
id	SERIAL	PK	Identifiant unique
name	VARCHAR(100)	UNIQUE	Nom de la source
url	TEXT	NOT NULL	URL racine
status	BOOLEAN	DEFAULT TRUE	État (Actif/Inactif)

6.3 Maquettes et Prototypes

6.3.1 Wireframes

Schémas filaires pour la page d'accueil (barre de recherche, filtres latéraux, liste), la page de détail (informations clés, description, bouton postuler), et le dashboard utilisateur.

6.3.2 Mockups

Maquettes haute fidélité respectant la charte graphique (couleurs modernes, typographie lisible) réalisées sur Figma.

7. CONTRAINTES DU PROJET

7.1 Contraintes Temporelles

- **Durée totale** : 14 semaines.
- **Jalons** : S1 (Kickoff), S4 (Ingestion finie), S7 (Backend fini), S9 (Frontend MVP), S14 (Livraison).

7.2 Contraintes Budgétaires

- **Budget** : Limité (Projet étudiant).
- **Hébergement** : Utilisation de tiers gratuits (Free Tier Render/Vercel) ou faible coût.

7.3 Contraintes Humaines

- **Équipe** : 4 étudiants.
- **Compétences** : Python/FastAPI, React/TS, SQL, Docker.

7.4 Contraintes Légales et Réglementaires

- **RGPD** : Respect de la protection des données utilisateurs (droit d'accès, suppression).
- **Droits d'auteur** : Respect des `robots.txt` des sites sources et citation des sources originales des offres.

7.5 Contraintes Techniques

- **APIs Tierces** : Respect des Rate Limits (ex: Adzuna).
 - **Stabilité des sources** : Les sites HTML (Jobartis) peuvent changer de structure, nécessitant une maintenance des parsers.
-

8. MÉTHODOLOGIE DE DÉVELOPPEMENT

8.1 Choix de la Méthodologie

Méthodologie retenue : Agile / Scrum. **Justification** : Permet une adaptation rapide aux changements (ex: structure d'un site source), une livraison incrémentale (MVP d'abord), et une meilleure gestion du travail en équipe.

8.2 Organisation du Travail

8.2.1 Phases du Projet

- 1. **Phase d'Analyse (S1)** : Analyse des besoins, PoC technique.
- 2. **Phase de Conception (S1-S2)** : Architecture, Modélisation UML, DB.
- 3. **Phase de Développement (S2-S10)** :
 - Ingestion (S2-S4)
 - Backend API (S5-S7)
 - Frontend MVP (S8-S9)
 - Matching/Alertes (S10)
- 4. **Phase de Tests (S11-S12)** : Tests unitaires, intégration, correction bugs.
- 5. **Phase de Déploiement (S12)** : Mise en production, CI/CD.
- 6. **Phase de Documentation (S13-S14)** : Rapport, Slides, Doc technique.

8.2.2 Sprints

- **Durée d'un sprint** : 2 semaines.
- **Nombre de sprints** : 7 sprints.
- **Cérémonies** : Weekly sync, Sprint planning simplifié.

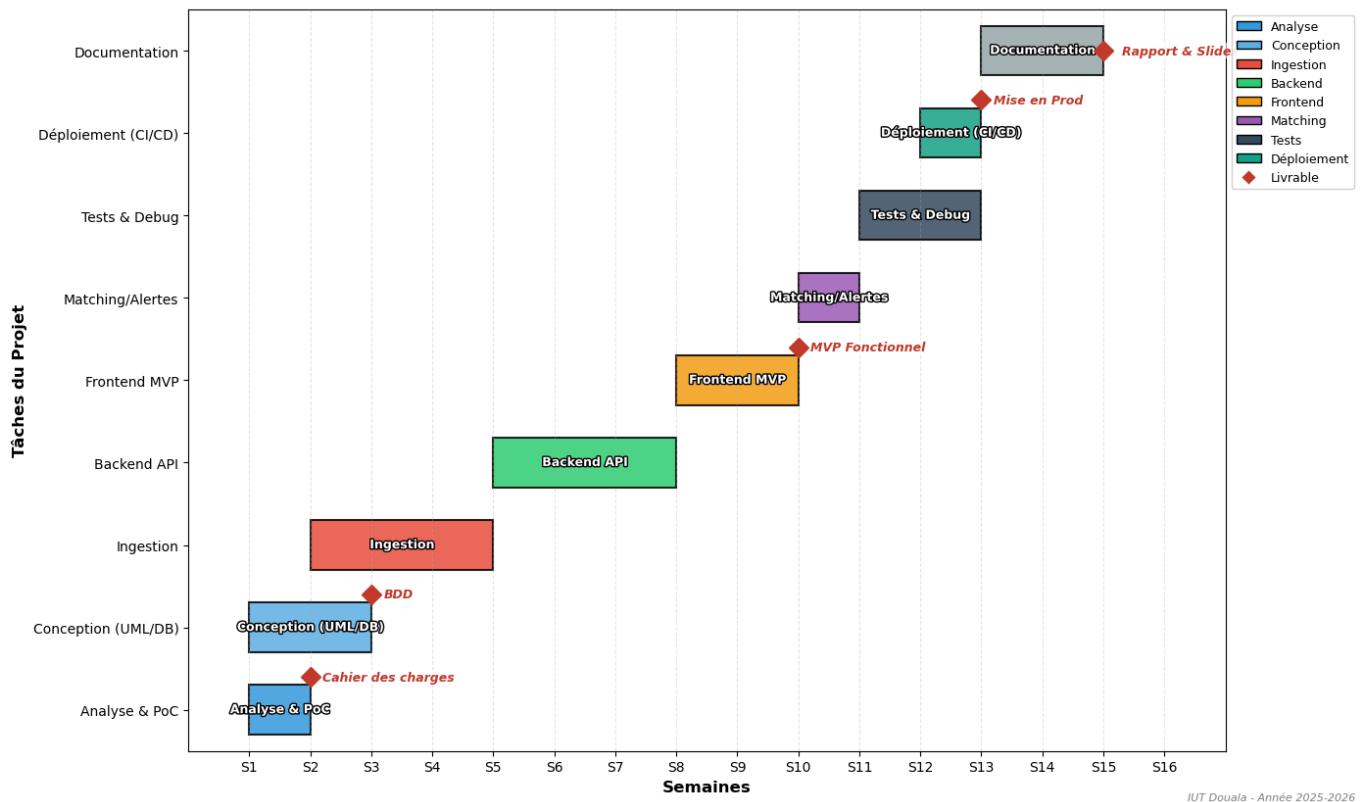
8.3 Livrables Attendus

Livrable	Description	Date de Livraison
Cahier des charges	Document de spécification (ce document)	S2
Code source	Repository GitHub (Backend + Frontend)	S14
Base de données	Scripts SQL / Dump	S14
Application déployée	URL accessible de la plateforme	S14
Documentation technique	Guide d'installation et API Docs	S13
Rapport final	Mémoire de fin d'études	S14

9. PLANIFICATION DU PROJET

9.1 Diagramme de Gantt

**Diagramme de Gantt - SYSTÈME DE RÉCUPÉRATION DE DONNÉES
(Plateforme d'Agrégation d'Offres d'Emploi)**



9.2 Découpage en Tâches

Sprint 1-2 : Ingestion & Base

- Mise en place environnement & Docker.
- Création DB PostgreSQL.
- Développement Parsers (Remotive, Adzuna).

Sprint 3-4 : Ingestion Avancée & Dedup

- Développement Parsers HTML (Jobartis, Emploi.cm).
- Implémentation Algorithme Dédoublement (Hash + Fuzzy).

Sprint 5-6 : API Backend

- Auth JWT.
- Endpoints Jobs (Search, Filter).
- Endpoints Watchlists.

Sprint 7-8 : Frontend

- UI Components (Tailwind).
- Pages Search, Details, Login.
- Intégration API.

9.3 Jalons (Milestones)

- 1. **Jalon 1 (S4)** : Pipeline d'ingestion fonctionnel (5 sources).
- 2. **Jalon 2 (S7)** : API Backend complète et documentée.
- 3. **Jalon 3 (S10)** : MVP complet (Front + Back + Alertes).
- 4. **Jalon 4 (S14)** : Version finale testée et déployée.

10. GESTION DES RISQUES

10.1 Identification des Risques

ID	Risque	Probabilité	Impact	Gravité
R1	API Adzuna limitée (Rate limits)	Élevée	Moyen	Moyenne
R2	Changement structure sites HTML	Moyenne	Élevé	Élevée
R3	Volume d'offres variable (Perf)	Moyenne	Moyen	Moyenne
R4	Retard dans le développement	Élevée	Élevé	Élevée

10.2 Stratégies d'Atténuation

Risque R1 : API Adzuna limitée

- **Prévention** : Mise en cache des résultats.
- **Mitigation** : Respect strict des quotas, clés multiples si nécessaire.

Risque R2 : Changement structure sites HTML

- **Prévention** : Scrapers modulaires et isolés.
- **Mitigation** : Monitoring des erreurs de parsing (Sentry) pour réagir vite.

Risque R3 : Volume d'offres variable

- **Prévention** : Indexation optimisée (tsvector) dès le début.
- **Mitigation** : Pagination efficace côté API.

Risque R4 : Retard dans le développement

- **Prévention** : Suivi régulier (Sprints).
- **Mitigation** : MVP strict défini (Périmètre V1), réduction des features secondaires ("Nice to have").

11. CRITÈRES D'ÉVALUATION ET DE VALIDATION

11.1 Critères de Réussite du Projet

Le projet sera considéré comme réussi si :

- 1. L'ingestion automatique fonctionne pour les 5 sources cibles.
- 2. Le dédoublonnage est effectif (pas d'offres identiques visibles).

3. La recherche est fonctionnelle et rapide (< 200ms).
4. Les alertes sont envoyées correctement par email.
5. Le code est propre, testé et déployable via Docker.

11.2 Tests et Validation

11.2.1 Tests Unitaires

- Backend : Pytest pour les services et utilitaires.
- Frontend : Jest pour les composants critiques.

11.2.2 Tests d'Intégration

- Vérification de la chaîne Ingestion -> DB -> API.

11.2.3 Tests Fonctionnels

- Validation des parcours utilisateurs (Recherche, Alerte).

11.2.4 Tests de Performance

- Vérification des temps de réponse de l'API sous charge modérée.

11.3 Recette Finale

- Validation par l'encadreur académique.
- Démonstration fonctionnelle de la plateforme.
- Revue du code et de la documentation.

12. PERSPECTIVES ET ÉVOLUTIONS FUTURES

12.1 Évolutions Envisagées

1. **Alertes Telegram/WhatsApp** : Ajout de nouveaux canaux de notification.
2. **IA de Matching** : Utilisation de NLP avancé pour mieux matcher offres et profils.
3. **Application Mobile** : Version React Native.

12.2 Limitations Actuelles

- Alertes uniquement par Email pour la V1.
- Pas de candidature directe sur la plateforme (redirection vers le site source).

12.3 Opportunités d'Extension

- Partenariats avec des entreprises locales pour diffusion directe.
- Analyse de données pour fournir des statistiques sur le marché de l'emploi.

13. ANNEXES

Annexe A : Glossaire

Terme	Définition
Scraping	Extraction automatique de données d'un site web.
Parsing	Analyse syntaxique d'un contenu pour en extraire les informations structurées.
Fuzzy Matching	Comparaison approximative de chaînes de caractères.
MVP	Minimum Viable Product (Produit Minimum Viable).

Annexe B : Références Bibliographiques

1. Documentation FastAPI : <https://fastapi.tiangolo.com/>
2. Documentation React : <https://react.dev/>
3. PostgreSQL Full Text Search : <https://www.postgresql.org/docs/current/textsearch.html>

Annexe C : Normes et Standards Utilisés

- **REST** : Architecture pour les services web.
- **PEP 8** : Guide de style pour le code Python.
- **SemVer** : Gestion sémantique de version.

Annexe D : Acronymes

Acronyme	Signification
API	Application Programming Interface
SGBD	Système de Gestion de Base de Données
UML	Unified Modeling Language
JWT	JSON Web Token
SPA	Single Page Application

VALIDATION DU CAHIER DES CHARGES

Membres du Groupe :

Signatures : _____

Date : _____

Encadreur Académique :

Signature : _____

Date : _____

Encadreur Professionnel (si applicable) :

Signature : _____

Date : _____

Document rédigé conformément aux standards académiques internationaux
Institut Universitaire de Technologie (IUT) Douala
Année Académique 2025-2026